HealthLedger — Complete Project Writeup

Structure: Abstract · Introduction · Literature Review · Methodology · Results · Discussion · Conclusion · References

---

Abstract

This project presents **HealthLedger**, a blockchain framework for a secure, distributed healthcare system that gives patients ownership and control of their medical data while keeping it verifiable and tamper-proof across providers. The framework combines four pillars: **(1)** a permissioned Proof-of-Authority (PoA) consensus that keeps the ledger consistent without energy-intensive mining, **(2)** asymmetric hybrid encryption (AES-256-GCM wrapped with RSA-2048 keys) so records are stored as ciphertext off-chain while only SHA-256 hashes and metadata live on-chain, **(3)** patient-controlled consent smart contracts that enforce grant/revoke access at runtime, and **(4)** role-based access control for all stakeholders — Patient, Doctor, Nurse, Hospital, Laboratory, Insurance, and Admin. It ships as a secure REST API, a responsive web UI (React/Vite), and an Expo mobile app, and is deployed live on GitHub Pages and Render. Measured results show ~1.2 ms full-chain integrity validation, ~4 ms record verification, ~1 ms PoA block sealing, instant consent enforcement, and immediate detection of tampering.

Introduction

Healthcare data today is fragmented across hospitals, clinics, laboratories, and insurers. Each organization maintains its own silo; records are inconsistent, slow to share, and vulnerable to tampering and unauthorized access. Patients have little visibility into who accesses their data or how consent is managed — consent is typically a manual, paper-based process that can take days. Privacy regulations (HIPAA/GDPR-style) demand auditability and patient control.

Blockchain offers a **single, immutable, shared source of truth** with an automatic audit trail. But public blockchains based on proof-of-work are energy-intensive and expose data by design, while naive deployments store plaintext on-chain. The objective of this project is therefore to design and implement a framework that is:

- **Permissioned** — only trusted, validated institutions participate in consensus (a fixed validator set, no energy mining);
- **Confidential** — data never leaves as plaintext; encrypted payloads live off-chain and hashes live on-chain;
- **Patient-centered** — patients grant and revoke access instantly, enforced in real time on every read;
- **Complete** — a working web app, a mobile app, and a live deployment rather than a paper-only design.

Literature Review

- **Centralized EMR/EHR systems** (e.g. Epic, Cerner) are mature but provider-centric, exhibit poor cross-institution interoperability, and give patients no practical control over consent.
- **MedRec (Azaria et al., 2016)** at MIT pioneered blockchain-based medical record access management using a permissioned ledger with smart-contract "data structures" for patient–provider relationships. Its consent/relationship layer is the direct intellectual ancestor of this project's consent registry.
- **Healthcare data gateways (Yue et al., 2016)** introduced blockchain-anchored gateways that control data sharing and preserve patient privacy, establishing the pattern that access policy can live on-chain while the data itself stays off-chain.
- **Hash-anchored off-chain storage** (public cloud/IPFS plus an on-chain hash) is the established integrity pattern for enterprise data. This project adopts the pattern but encrypts the off-chain payload, so even the storage location holds ciphertext only.
- **Permissioned consensus over PoW**: Zheng et al. (2017) surveyed blockchain consensus and showed permissioned models (PBFT/Raft/PoA) eliminate energy cost and provide deterministic finality. **Proof-of-Authority (Wood, 2015)**, adopted by Quorum/Clique, suits enterprise healthcare because validators are known, licensed institutions rather than anonymous miners.
- **Guardtime's Estonian e-health KSI blockchain** demonstrated national-scale, on-chain integrity anchoring without exposing the health records themselves — validation that hash-anchoring scales in production, which this framework mirrors at application scale.
- **Ancile (Dagher et al., 2018)** showed fine-grained, patient-mediated access control for EHRs using permissioned blockchain contracts, reinforcing the consent-contract approach used here.

Methodology

Architecture

The system comprises three deployment artifacts served by one backend API:

1. **Server** — Node.js + Express (`server/`), exposing the REST API, the PoA ledger, the consent registry, and file/off-chain storage.
2. **Web client** — React + Vite (`client/`), built to `docs/` and served by GitHub Pages.
3. **Mobile app** — Expo / React Native (`mobile/`), consuming the same API.

1. Proof-of-Authority consensus (`server/blockchain.js`)

- A fixed, permissioned validator set: `VALIDATOR-CARE-NODE-A`, `VALIDATOR-CARE-NODE-B`, `VALIDATOR-LAB-NODE`, `VALIDATOR-INSURANCE-NODE`, `VALIDATOR-HOSPITAL-NODE`.
- Blocks are sealed by the validator currently in rotation. Each block is HMAC-SHA256 signed with a validator-only secret (`VALIDATOR_SECRET`).

- `isChainValid()` recomputes every block hash, checks previous-hash linkage, and re-verifies every validator signature.
- Administrators seal pending transactions via `POST /api/mine`; the next validator in rotation signs.

2. Hybrid asymmetric encryption (`server/crypto.js`, `server/offchain.js`)

- Each record is serialized to JSON, hashed with SHA-256 (the on-chain `dataHash`).
- The plaintext is encrypted with **AES-256-GCM**; the ephemeral AES key is wrapped with **RSA-2048 (OAEP-SHA256)** using a keypair auto-generated to `server/keys/` at first boot.
- The ciphertext is written to the off-chain store (`server/data/offchain`); **only** `dataHash`, `dataRef`, `patientId`, `author`, `contentType`, `timestamp` go on-chain.
- On read, the payload is decrypted and re-hashed; the result must equal the on-chain `dataHash`, otherwise the record is rejected (decrypt-then-hash integrity check).

3. Patient-controlled consent smart contracts (`server/consent.js`)

- A ConsentRegistry implements grant/revoke/list with structured provider records (name + type).
- **Doctor-type** consents are scoped to the named doctor; **organization-type** consents role-match (Hospital covers hospital and nurse users; Laboratory covers laboratory users; Insurance covers insurance users).
- Every record and file read passes through `hasActiveConsent()`; a revoked grant blocks access instantly. Contract actions are themselves recorded on the ledger, forming an audit trail.

4. Role-based access control (`server/auth.js`)

- JWT-authenticated roles: **Patient, Doctor, Nurse, Hospital, Laboratory, Insurance, Admin**.
- Write operations are limited to Doctor/Nurse/Admin/Hospital/Laboratory; Insurance is read-only.
- Patients read only their own records; staff reads are consent-gated; Admin bypasses.

5. Seeded demonstration data

- 110 patients (P-2001…P-2110) across 11 departments, each with a medical record, a report image, a login account, and seeded consents for their physician and the hospital organization.
- 22 department doctors with consent-gated access to their own patients.

Results

All results were measured on the live system (2026-09-05); full reproduction steps are in `PERFORMANCE_EVIDENCE.md`.

Metric	Local node	Deployed (HTTPS,

		cold)
Full-chain integrity validation (~176 txs, 4 blocks)	**~1.2 ms**	~0.22 s
Record retrieval + decrypt + hash-verify + parse	**~4 ms**	~0.20–0.32 s
PoA block sealing	**~0.95 ms**	—
Consent grant API (incl. client overhead)	**~23 ms**	—
Consent revocation	Instant (lab → **403**)	Instant
Consent grant	Instant (lab → **200**)	Instant
Tamper detection	Chain valid ⇒ altered tx hash ⇒ **invalid**	—

Access-control verification (live):

Scenario	Result
Physician accessing their own patient	✅ 200, records decrypted + verified
Physician accessing another physician's patient	✅ 403 (no consent)
Nurse / Hospital on a consented patient	✅ 200
Laboratory before patient grant	✅ 403
Laboratory after patient grant	✅ 200
Laboratory after patient revoke	✅ 403
Insurance read	✅ 403 without consent / read-only role
Patient viewing own records	✅ 200

Discussion

The measured results demonstrate that a permissioned, hybrid-encrypted design can deliver both **confidentiality** and **verifiability** at interactive speed: a full-chain audit is a millisecond-scale operation rather than a departmental reconciliation task, and consent enforcement is instantaneous compared with days of manual paperwork. These timings provide the evidence base behind the report's overhead-reduction projections (e.g. "25–30% lower administrative overhead") — they should be presented as projections justified by the measurements, not as a controlled before/after study.

Design trade-offs.

- *Single-node deployment, virtualized validators.* The deployed backend runs one instance whose five validator identities are rotated in software. The architecture is permissioned and signature-checked exactly as a multi-institution network would be, but genuine geographic/legal separation of validator nodes is future work.

- *Consensus vs. decentralization.* PoA trades decentralization for finality, low latency, and accountability — the correct trade for regulated healthcare, where validity derives from trusted institutions rather than anonymous competition.
- *Off-chain store.* The off-chain payload store is the trust root for record content; confidentiality relies on the held RSA keypair, so key management (HSM/cloud KMS) is the recommended production hardening.
- *SQLite for file metadata.* Sufficient for the demonstration; a production audit table would migrate to a managed database.

Limitations. Multi-institution consortium deployment, GDPR-style data erasure semantics (tamper-evident ledgers and deletions are in tension), and formal smart-contract verification are outside the current scope.

Conclusion

HealthLedger shows that a healthcare blockchain need not be either insecure or energy-inefficient. By combining permissioned Proof-of-Authority consensus, hybrid asymmetric encryption with off-chain storage, on-chain hash anchoring, patient-controlled consent contracts, and complete role-based access control, the framework delivers a confidential, verifiable, and patient-centered record system that works in practice — proven by a live web app, a mobile app, and measured sub-millisecond-to-millisecond operations. The work validates the report's core claims and provides a concrete, reproducible foundation for multi-institution healthcare networks.

References

1. Azaria, A., Ekblaw, A., Vieira, T., & Lippman, A. (2016). *MedRec: Using Blockchain for Medical Data Access and Permission Management.* 2nd IEEE International Conference on Big Data (BigData 2016).
2. Yue, X., Wang, H., Jin, D., Li, M., & Jiang, W. (2016). *Healthcare Data Gateways: Found Healthcare Intelligence on Blockchain with Novel Privacy Risk Control.* Journal of Medical Systems, 40(10), 218.
3. Dagher, G. G., Mohler, J., Milojkovic, M., & Marella, P. B. (2018). *Ancile: Privacy-preserving framework for access control and interoperability of electronic health records using blockchain technology.* Sustainable Cities and Society, 39, 283–297.
4. Zheng, Z., Xie, S., Dai, H., Chen, X., & Wang, H. (2017). *An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends.* IEEE International Congress on Big Data, 557–564.
5. Wood, G. (2015). *Ethereum: A Secure Decentralised Generalised Transaction Ledger.* Ethereum Yellow Paper.
6. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*
7. Androulaki, E., et al. (2018). *Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains.* EuroSys 2018.
8. Mettler, M. (2016). *Blockchain Technology in Healthcare: The Revolution Starts Here.* IEEE 18th International Conference on e-Health Networking, Applications and Services (Healthcom).

9. Guardtime. *Estonian e-Health Blockchain (KSI) Proof-of-Client Implementation.*